

Player Legacy as a Persistent and Emergent Progression Mechanism in Rogue-Lite Games: Independent Project in Gaming

Christos Ringas
University of Gothenburg
gusringach@student.gu.se

1 Abstract

This study explores the idea of player legacy as a developing system that reforms the game based on the player's deaths and their previous characters' actions, while populating the dungeon with ghost instances of the player's character during these failed attempts. This approach is concerned with the development of an input recording system capable of recording the exact player inputs, storing them in local persistent data, and then pushing these traces into virtual devices where the ghost players would execute each action exactly per timestamp. This creates the illusion of playing against another player while, in reality, you are against your own character's actions.

2 Introduction

Numerous games have been developed around the concept of play legacy to create interesting game interactions. A prevalent genre around this area has always been the rogue-lite games [4], where the dungeon evolves based on the players' previous runs. In some cases, when failing the run, the player will have to start again with a new character and face similar challenges, while also dealing with the dungeon being in a new state. In this project, a recording system is developed to showcase ways of changing the state of the game, deriving from the player's actions during previous failed attempts. So, the player ends up failing their run, and when starting over again, they will face their previous characters and actions as an additional obstacle to success, but at the same time have the opportunity to retrieve lost equipment.

In contrast to other games that instead adapt to the player's play style, the player encounters a persistent and explicit duplicate of their past selves' path, the characters themselves, and especially their identical movements and actions as part of the dungeon. Moreover, the dungeon generates "ghosts" players performing these actions according to the exact inputs they previously executed by passing them into virtual input devices that read the corresponding trace. After verify-

ing that the recording system can record these actions and pass them into the virtual devices, one can use this system to develop and instantiate different variants of these ghosts, giving the player the illusion of playing alongside somebody else. Some of these variants can interfere with the player's goal, making the game more difficult by blocking the player's way, requiring them to be dealt with to proceed, or, in other cases, they can also act as friendly NPCs helping the player progress.

3 Background

Before exploring different approaches around the idea of player legacy in gaming, a definition of "player legacy" must be considered in this context. The term is interlinked with player agency and their ability to have the main role in defining their future based on their past, since it is about the player being able to have some control. The first part, "player" of the term, indicates agency towards their own narrative and how they contribute to this chain of progression. For instance, this is something that can be found in narrative games, cybertexts, and Player-Generated narratives such as Undertale [7], with player choices serving as a mechanism in experiencing and unraveling the different parts of a game's story in different ways based on the player's decisions and interpretations [11]. On the other hand, "Legacy" refers to something from the past, a heritage that continues in the later stages of the game that could also exist in its own time and place outside of a current playthrough. Another way to define legacy in rogue-lites is the idea of investment into these different "Avatars" that the player will use during their playthroughs to achieve their goal. In this sense, players utilize the available tools via these short-lived avatars/characters that will serve towards ultimate success [1].

Rogue-lite games and their sub-genres have implemented relevant mechanisms evolved around the permadeath [9] mechanism that binds the idea of the player's legacy as a structural method to reform their dungeons. What the player's character may leave behind can be helpful or harmful for future runs. This meta-progression as-

sists the player in the next run, engaging them more in the game and creating a sense of achievement even in the case of failed attempts. For instance, in games such as Enter the Gungeon [5] and Hades [10], players can purchase unlockable items and upgrades that will help them progress through future runs [1]. Also, Rogue Legacy [2] utilizes a hereditary system involved with the narrative of the game, with the future playable characters being descendants of the previous one. It is an interesting approach, continuing the narrative of the game and future runs that affect the descendants' good and bad traits in accordance with the player's previous playstyle, playing as their predecessors, not only affecting their current playthrough but also future generations.

Another example that is more associated with the concept described here is the example of "bone file" found in rogue-like games such as NetHack [8], where a previous state of a dungeon has a probability of being saved for the player to encounter it in later runs. In this example, the player may have to fight their previous characters in their new form of a hostile ghost [1]. In addition, if the player overcomes the ghost character, they could get their previous equipment back among the successive characters.

On the contrary, other genres are also associated with the player's playstyle. For instance, in Echo [12] and Hello Neighbor 4 [6], the enemies will learn from the player's playstyle and mimic their behavior or adapt based on these, making their progression more challenging [16]. However, Echo adapts based on the player's behavior but does not replay their exact actions. A sophisticated system in its technical aspect, but it does not portray the feeling of fighting your past self, and this is a topic worthy of further investigation.

4 Discussion

Throughout the research and development processes of the project, several views and considerations were explored around the idea of player legacy, with the most important being a persistent method for tracking player inputs and their development. This input recording approach is the main mechanism for creating the reformed environment for the player who will encounter their legacy in each playthrough. One consideration was the appearance of the playable character and how they should be identifiable by the player in future runs. Lastly, showcasing the concept was the final concern since some other core gameplay mechanics must be present to drive this idea of recording and playback actions in an interesting and playable way. In this section, we are analyzing the design and development process of the game systems while also evaluating them.

4.1 Game Systems

4.1.1 Gameplay

The example game to showcase the system is a simple 2D side-scrolling platformer where the playable character uses a close-combat weapon to fight their enemies in their attempt to finish the dungeon. There is a single standard enemy type in the game, which is the "monster," and it will attack the player when spotted [17]. The player's goal is to progress through the short dungeon by defeating the enemies that populate the combat areas. A vital aspect of the gameplay in this project is designing a fighting style where the player can attack the standard monsters, but most importantly, fight the ghosts successfully. The player and ghosts should be able to co-exist in the same space and create the illusion of fighting each other based on the ghosts' sequence.

For that reason, the player and at the same time the ghosts, following the player's logic, can perform three close combat attack variations, with these being a standard weak attack, that can also be performed while moving, a "heavy" larger attack, and a "low" thrust attack. The latter attacks cannot be performed while moving. In addition to these actions, the player and ghost can perform jump and block actions. With this set of actions, the player has some variations in their attacks combined with movement that could potentially generate interesting sequences later on for the ghosts to replicate.

As a result, the player can fight the ghost, and the ghost gives the impression of fighting back while executing the actions in the trace. Moreover, the player can also use different variations of weapons that have their own variations of the three attacks, following the same template. To use them, they need to find the weapons in the dungeon or be lucky enough to have the weapon dropped by the defeated ghosts. Finally, health orbs can be found in the dungeon to replenish their health meter.

4.1.2 Modular Character

Trying to deal with the idea of identification of the different characters per playthrough, it was decided to use a modular 2D character template that can have each of their body parts isolated from the others. For that reason, the character should be created so that each of these body parts is animated separately. Essentially, the character is divided into five different body parts that consist of the *body*, the *hair*, the *eyes*, the *mouth*, and the *weapon*. These parts create a playable character template that can utilize distinct facial characteristics, making them recognizable from the rest of the ghosts, including the player, and also make it possible to consistently follow the animations required during each state of the character controller.

With this model in mind, the animations for each body part variation were created using Aseprite. They were exported as separate sprite sheets that were then pushed through the Unity engine to be processed through the state machine in the animator. For the modular system to perform as intended, the same number of frames per body part per state is required. But also, transitioning each body part to the next state simultaneously, and playing each animation from the start when a change happens, is another requirement. Eventually, the animator could change each of the animated parts in real time and make sure that everything is synchronized. This means that each of the states has a sprite sheet per body part. This allows the character to be able to change their characteristics and make them more unique, but at the same time, they can change weapons and transition correctly to the current state by updating each of the body parts' sprite [13].

4.1.3 Recorder

The recorder is the system used to record the player's inputs into a tracer file for the ghost to read. The new input system of Unity was used to read and perform actions from the input devices [15]. Following this component structure, the built-in recorder API was also used to record the traces of the input actions, which are then stored in the persistent data location in the local storage of the machine running the game [14].

The complete process in a high-level operation is as follows: First, the recorder is activated to record every input event, such as buttons pressed, mouse position, from all the available devices. After that, when the recording process has stopped, these data are stored locally into an InputRecording trace with all the actions executed, together with their timestamps. Then, this trace can be used again through the same input system and specifically the available input devices, so the player or the GameObject that uses the input system component can perform the recording's actions based on timestamps. In this way, the GameObject performs the same actions found in the trace through the same input bindings and in the same way as the player did when actually performing the actions. In the current project, this trace was played in a virtual device instead of the one the player is using, with each of the virtual devices assigned to a trace and isolated from the player's actual inputs.

Use cases of the recorder:

- When something interesting happens, based on checks, there is a chance that the player's input will be recorded and stored. Then these records can be replayed by an instance of the same character, but this time targeting the player. This is a "ghost" player with the character's previous

appearance, name, equipment, etc. These recordings can be location-specific, replayed from the same starting position, or instantiated in a given position, making them more unexpected.

- The Ghost can be a friendly NPC with the appearance of a previous player's character that may help the player through an event they have already encountered in previous attempts.
- Defeating a ghost does not mean that they have disappeared. instead, their data is stored and persists through the local storage and can be instantiated at any moment.

4.2 Planning

The MoSCoW table 1 in Appendix A1 shows the prioritization method used for the core game features, while the backlog table 2 in Appendix A2 was used to keep track of the tasks that needed to be completed.

4.3 Early Considerations and Challenges

- The fighting style should be sufficient for demonstration, challenging, and simple for the Ghosts' characters to apply.
- Ghosts should be identifiable. Perhaps using a randomizer for cosmetics applied to every new character.
- Simultaneous ghosts must have a limit. Otherwise, the game can feel too chaotic or unplayable.
- The player should fail for the ghosts to appear in the following runs, but it should also be balanced.
- The Technical aspect and implementation of the recording system and playback of the trace into the ghost will be the hardest part.
- The spawn rate and locations of the Ghosts must be considered to balance out fights among the current player and stronger Ghosts. Perhaps a Ghost grading system can be associated with their spawning location in the dungeon.

4.4 Working Towards some of the Challenges

During the research on the recorder, there were discussions about the idea of recording and replaying the input actions of the player for creating a video capture using the Unity Recorder [3]. Some of the users struggled with unofficial versions of the recorder input system, having a lot of inconsistencies. However, with the updated official release that Unity announced, at the time, it was solving most of the problems they had before. The same API for the recorder was used here

since it seemed a good, handy tool and a reliable way to implement the recorder system needed for the project. Nevertheless, the API was challenging to work with and came with certain limitations, and issues emerged. One example was the unsupported functionality specifying which input device the recorder should record from. This was a feature only available in later versions and was not used in this case, since it required a later version of the game engine, which, due to the limited time-frame spent in development, was decided to bypass the upgrade that would have caused additional challenges for the rest of the project, taking away time from other features. So, the recorder was recording from any available device.

Related to this issue was the fact that the trace could not ignore the player's device. This means that the recorder was playing through any available player input, the ghosts' virtual device, or the player's input, giving the unwanted result of the player being controlled by the trace alongside the ghost. To solve this issue temporarily and only when creating and assigning the new virtual device to the corresponding ghost, the player's device is disconnected from the input system for this frame. This workaround came with limitations and new issues, since we are disconnecting the player device, a slight stuttering appears when this happens, but it is not very noticeable when there are only two or three ghosts on the screen. This method occurs for each trace, with a new virtual device being created and assigned to it. So, every time a new recording is playing due to the connections and disconnection of the player device, this phenomenon gets more noticeable. Yet, this works since by the time the ghosts get their trace and their virtual device, the player can continue right after. Lastly, to make sure that the system is not populated with unused virtual devices, there is a cleanup step after the trace has finished, deleting the device.

4.5 Using the Trace on the Ghost

The recording system records the player's inputs into a trace file in the local storage, but it also spawns the ghosts in the dungeon. Two variations of the ghosts are implemented, one being a combat ghost that appears only during the combat areas and the other being the quick ghost that appears randomly next to the player at any time.

In the case of the combat ghost, this consists of multiple and successive instances of the player's recordings found in the persistent data folder. Basically, the system requests the spawning of a combat ghost with a certain number of recordings. Then, by traversing all the run folders in the data location, one random run folder is selected that fulfills the requirements of the requests. In the run folder, the traces of each recording can be found together with the metadata regarding

each recording's starting player's position at the time the recording took place. This is important since using this metadata, location-sensitive recording can be played by checking against the collision component of the corresponding combat area. As a result, the correct recordings are chosen while the player was previously performing actions in this area, ensuring that the ghost executes the actions and interacts with the environment the same way as the player did in previous runs for this location. At each recording finish, the next recording in the list is activated. Lastly, the run folder also includes a ghost JSON file regarding the body parts indices and the character name, so the ghost would have the same appearance and name when spawned. Upon defeating them there is a change that they will drop their weapon.

In the case of the quick ghost, these are simpler, and any recording can be chosen from the runs that fulfill the requirements. They spawned next to the player's location, executing their recording for a shorter period, and have the role of a surprise ghost that the player should keep in mind. Also, they are not able to target, and no reward is provided by them.

4.6 Social, Ethical and Accessibility Considerations

Designing this kind of game raises certain social considerations due to its difficulty. The player starts the game while using the weakest weapon, making their way towards better weapons in the dungeon. However, in some cases, they will have to face some instances of themselves as ghosts that carry better equipment, such as a weapon that deals much more damage in comparison to the one the player uses. This creates some frustration for certain groups of players who are not used to this kind of behavior, with the difficulty level rising suddenly. To compensate for this, there is a chance for the player to be rewarded after defeating this difficult enemy by reclaiming their weapon. It is a balancing issue in cases where the players struggle in their first attempts, which could result in having to fight easier ghosts or the opposite. Although only ten recording slots are used currently in the prototype with the ghosts are getting refreshed. Another consideration comes from playing collectively with other people, with the player encountering other people's ghosts. Even though the particular prototype example did not have very sophisticated fighting sequences, comparing the sequences might create a feeling of competition among the players involved, which perhaps not everyone enjoys, ending in discouragement for some.

One Ethical consideration is the recording itself as a mechanic in the game. The game only records the necessary non-personalized data needed for its functionality, which can not result in the identification of the player

connected to a sequence, unless they also enter their name in the name field. The mechanic requires a form of key logging and writing to memory derived from the user interface, which can be either the keyboard or the controller used by the player. In this case, the player must be aware of such functionality before playing the game and should be presented with an early disclaimer screen that they are required to read and agree to before playing. Minimizing the data stored specifically on the player machine should also be considered, as even in early iterations, although the recordings were not large in file size, many of them could result in unnecessary space occupation on the player's local storage. Having a method for clearing the recording data from the player's side would also be helpful. Lastly, the theme of self-fighting requires special care, since it could sometimes be related to self-destructive themes that should be avoided or included as a disclaimer message of harmful themes.

Regarding the accessibility part in the game, one thing that was considered in the development was the visuals of the ghosts that needed to somewhat be different and distinguishable in comparison to the playable character's visuals, since they were very similar most of the time. For that reason, the opacity attribute was utilized as a simple solution for prototyping. Also, visual and audio cues during the fight were considered. For example, in heavy attacks, the player can see the attack telegraphs of the charging swing of a sword, indicating an incoming attack towards the player. These kinds of cues are necessary in this kind of game for the best user experience.

5 Future Work

The workarounds discussed should be reconsidered when building the recording system, and a custom approach would be the next step in this project. This would guarantee more control over the issues that appeared when using the built-in recorder. At the same time, the latest versions of the input recorder in combination with the latest Unity builds would have given more options when defining towards the devices we should be recording instead of recording with every available device. After implementing a better version of the recording system, it would also be interesting to see more variations of the ghost being friendly instead of only being hostile towards the player.

6 Conclusion

In conclusion, this prototype of a 2D side-scroller dungeon crawler game has been implemented, showcasing the player's legacy via the input recorder and the instantiation of the ghosts performing the recorded ac-

tions. As a result, the player's previous characters and especially their exact actions are changing the dynamic of the dungeon by reforming it into a more immersive sequence than before. Player agency adjusts the dungeon difficulty and its progression in a more interesting and emergent way, and at the same time, the system gives the impression of playing against another player. Moreover, the persistent progression mechanism is bound to the previous player's direct actions instead of a behavioral adaptive system. However, challenges occurred during the development that were not expected and forced the direction of the project towards some workarounds for these issues, which also introduced some limitations. Lastly, regarding the development of the recording system, a more concrete and custom approach would have been a better option in this case, providing more control over the recording functionality.

References

- Cartlidge, J. (Ed.). (2025). *The rise of the roguelite: Inside a gaming phenomenon* (1st). CRC Press. <https://doi.org/10.1201/9781003415473>
- Cellar Door Games. (2013). Rogue legacy [Video game, rogue-lite, permadeath, meta-progression]. <https://roguelegacy2.com/>
- Community, U. (2019). *Recording and replaying input (forum thread)* [Discussion thread on Unity Engine forums]. <https://discussions.unity.com/t/recording-and-replaying-input/756993>
- contributors, W. (2025). Roguelike [Accessed: 2025-10-23]. <https://en.wikipedia.org/wiki/Roguelike>
- Dodge Roll. (2016). Enter the gungeon [Video game]. <https://dodgeroll.com/gungeon/>
- Dynamic Pixels / tinyBuild. (2019). Hello neighbor [Video game, stealth, puzzle, rogue-lite elements]. <https://www.helloneighborgame.com/>
- Fox, T. (2015). Undertale [Video game, role-playing, indie]. <https://undertale.fandom.com/wiki/Undertale>
- NetHack Dev Team. (1987). Nethack [Video game, roguelike, permadeath]. <https://www.nethack.org/>
- Parker, R. (2017). The culture of permadeath: Roguelikes and terror management theory. *Journal of Gaming & Virtual Worlds*, 9(2), 123–141. <https://doi.org/10.1386/jgvw.9.2.123.1>
- Supergiant Games. (2020). Hades [Video game]. <https://www.supergiantgames.com/games/hades/>
- Turley, A. C. (2018). *Reading the game: Exploring narratives in video games as literary texts* [Master's thesis]. Indiana University–Purdue University Indianapolis. <https://hdl.handle.net/1805/18520>

- Ultra Ultra. (2017). Echo [Video game, rogue-lite, meta-progression]. [https://en.wikipedia.org/wiki/Echo_\(2017_video_game\)](https://en.wikipedia.org/wiki/Echo_(2017_video_game))
- Unity Technologies. (2023a). Animatoroverridecontroller — scripting api [API Documentation, Unity Engine]. <https://docs.unity3d.com/ScriptReference/AnimatorOverrideController.html>
- Unity Technologies. (2023b). Class inputrecorder — unity input system api (v1.1) [API documentation for the InputRecorder component in Unity]. <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.1/api/UnityEngine.InputSystem.InputRecorder.html>
- Unity Technologies. (2025). Input system — input system — 1.15.0 [User manual for Unity Input System package]. <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.15/manual/index.html>
- Zhadan, A. (2018). *Artificial intelligence adaptation in video games* [Master’s thesis]. Linnaeus University. <https://urn.kb.se/resolve?urn=urn:nbn:se:lnu:diva-79131>
- zneeke. (2023). Dark fantasy monster pack 1 [Accessed: 2025-10-25]. <https://zneeke.itch.io/dark-fantasy-monster-pack-1>

A Appendix

A.1 MoSCoW Prioritization

Table 1: MoSCoW prioritization for core game features

Priority	Features
M – Must	• Player input recorder system • Permadeath • Basic combat system • Platforming controls (jump, 2D movement) • A playable character • A version of the player ghost
S – Should	• Meta-progression with the player being able to retrieve their previous equipment from fallen ghosts • Player legacy system with the persistent ghost recordings and their data being stored locally • Small Dungeon level • Regular enemies to fight • Modular appearance for the player and the ghost
C – Could	• Multiple weapon variations • Multiple ghost variations • Consumables • Boss fight • Friendly NPCs • A simple story • More complex Fighting style
W – Won't	• Multiplayer • Procedural dungeon generation • Character upgrading system

A.2 Development Backlog

Table 2: Development Backlog in chronological order

Category	Tasks / Description
Done	• Create animations in Aseprite and import them into Unity. • Re-export Aseprite sprite sheets with correct centering. • Build and synchronize the animation controller with scriptable objects for the modular character. • Integrate Unity’s new input system. • Add all character states in animator controller. • Design combat and fighting style logic. • Refine the modular body parts system and character customization tools. • Implement hitboxes with keyframe events for the attack actions. • Create health system and enemy testing ghost prefab. • Refactor the player movement logic. • Implement a basic input recording and replay system for “Ghost” mechanic using Unity Input Recorder. • Implement platforming improvements, including pass-through and drop-off platforms and camera follow. • Implement Ghost spawning basic logic. • Fix sprite layering across multiple entities. • Implement interaction system (doors, loot) and player UI. • Add loot and drop systems with randomization (e.g., 30% weapon drop from defeated ghosts). • Implement block mechanic and stamina UI meter. • Randomize player character appearance at run start. • Implement hostile/friendly Ghost variations (Combat and Quick ghosts). • Add standard enemy ”monsters” AI with patrol, attack, and environment checks. • Integrate lighting using mask-based techniques. • Expand cosmetic variety (eyes, mouth, hair) and weapons with different stats/appearance. • Implement player death. • Implement info UI panel for weapon stats in loot containers. • Add player name input field during run start and display above the player/ghost. • Disable pause during combat encounters. • Implement door unlock in the combat area after combat completion via enemies and ghost group detection. • Improve enemy detection logic (ignore player behind walls). • Spawn ghosts dynamically based on recording positional JSON metadata. • Add Quick Ghost spawning over time. • Add new default weapon (wooden spear). • Add music and sound effects. • Add health orb and randomized item drops.
Future / Not Implemented	• Add environmental interactions involving Ghost triggers (e.g., pressure platforms to open doors). • Expand on the enemy loot drop system after death (standard and Ghost enemies). • Implement stats system manipulated from passive and active items (health, damage, speed). • Store playable character stats persistently for the ghosts to utilize. • Add a hybrid Ghost-AI enemies type combining recorded input playback and AI logic (e.g., boss fights). • Performance optimization for Ghost spawning (reduce stutter and device pairing issues). • Implement ranged attacks or introduce new enemy archetypes. • Add temporary power-up drops (speed boosts, etc.).
Last To-Do List Implemented	• Finalize input recording scope (add checks for initiating the recording system). • Optimize ghost instantiations per combat area to minimize stuttering. • Adjust recording and ghost spawning frequency dynamically based on combat state. • Ensure unique Ghost groups will be spawned with no duplication on the same screen. • Implement level design elements (tiles, doors, enemy patrols, add black pool of death).

A.3 Game Figures

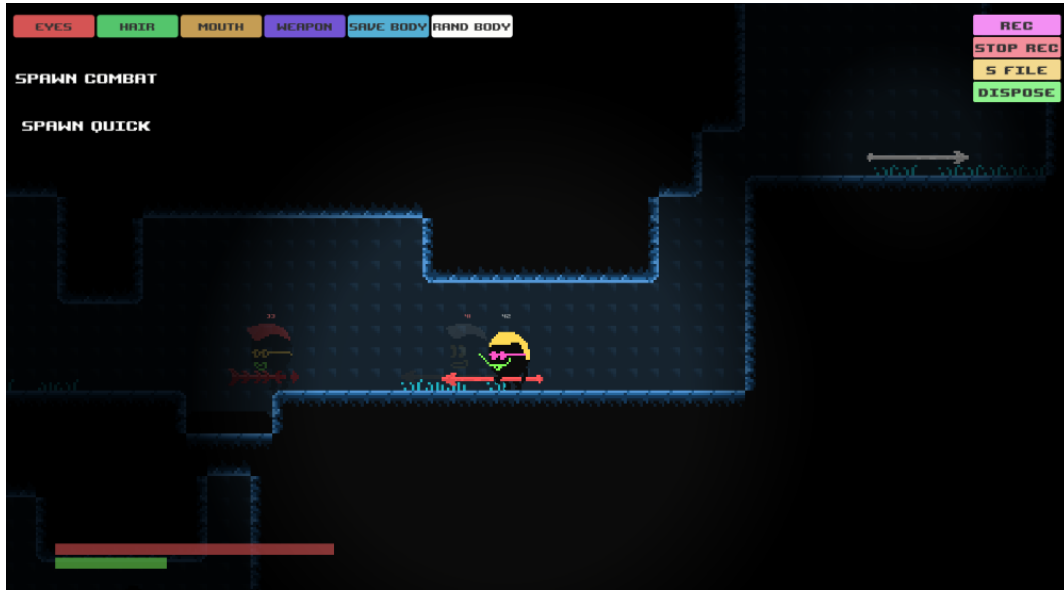


Figure 1: Image of the game, including the debug buttons for character customization and recording actions.

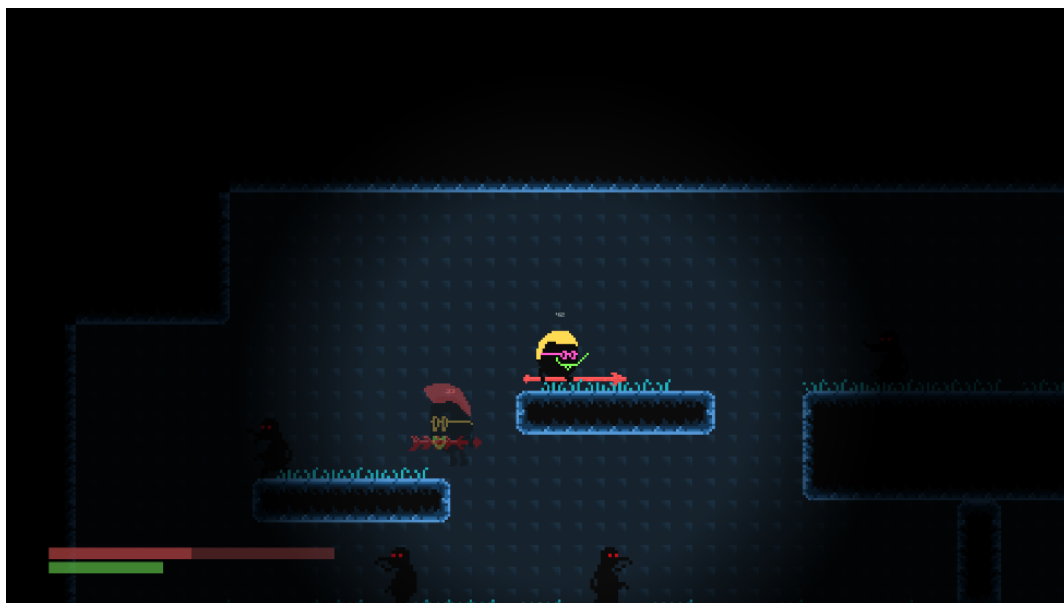


Figure 2: Image of the player inside the first combat area.